

Machine Learning Revision Plan (Before Restarting Deep Learning)

You already have one huge advantage:

- You have BUILT ML projects.

That means you are not starting from zero.

Right now, your problem is:

“I can use models, but I don’t deeply understand WHY they work.”

That is extremely common.

The goal of this plan is to transform you from:

- “I know how to use ML libraries” into
 - “I understand ML deeply enough to confidently learn Deep Learning.”
-

Main Goal

By the end of this revision plan, you should:

- Understand the intuition behind ML algorithms
 - Understand math used in ML at a practical level
 - Know when to use which model
 - Understand bias/variance/tradeoffs
 - Understand evaluation properly
 - Understand ensemble methods deeply
 - Understand optimization basics
 - Read ML papers/blogs without getting lost
 - Restart Deep Learning without confusion
-

IMPORTANT RULE

Do NOT just watch videos.

For every topic: 1. Learn intuition 2. Learn math lightly 3. Implement small examples 4. Explain it in your own words 5. Use it in a mini project

That cycle is what creates real understanding.

Total Timeline

Recommended:

- 6–8 weeks
- 1.5–2.5 hours/day

Since you already know some ML:

- Focus more on understanding than speed
-

PHASE 0 — Setup Your Learning System (1 Day)

Create These Folders

```
ML-Revision/  
|  
├─ notes/  
├─ implementations/  
├─ mini_projects/  
├─ datasets/  
├─ experiments/  
└─ cheatsheets/
```

Create a GitHub Repo

Suggested name:

```
ml-revision-journey
```

Document everything.

Employers LOVE seeing:

- experiments
 - notes
 - comparisons
 - mistakes
 - learning progression
-

PHASE 1 — Core Foundations (Week 1)

This is the most important phase.

If this phase becomes strong, everything else becomes easier.

1. What Machine Learning REALLY Is

Learn

- What is learning?
 - Features vs labels
 - Training vs testing
 - Generalization
 - Overfitting vs underfitting
 - Bias vs variance
 - Parametric vs non-parametric
 - Supervised vs unsupervised
 - Regression vs classification
-

You MUST Understand Deeply

Overfitting

What it REALLY means:

- Model memorizes patterns/noise
 - Performs well on training
 - Fails on unseen data
-

Bias vs Variance

Low bias:

- Learns patterns well

High variance:

- Sensitive to training data

This concept appears EVERYWHERE:

- trees
 - boosting
 - neural networks
 - regularization
-

2. Data Preprocessing

Learn

- Missing values
 - Encoding categorical variables
 - One-hot encoding
 - Label encoding
 - Feature scaling
 - Standardization
 - Normalization
 - Train/test split
 - Data leakage
-

IMPORTANT

Understand:

Why scaling matters

Especially for:

- KNN
- SVM
- Logistic regression
- Gradient descent

Trees generally do NOT require scaling.

This is an important interview question too.

3. Evaluation Metrics

Classification

Learn deeply:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- ROC-AUC

You should be able to explain:

“Why accuracy is bad for imbalanced datasets.”

Regression

- MAE
- MSE
- RMSE
- R^2

Understand:

- why squaring errors matters
 - why RMSE punishes large mistakes
-

Mini Tasks

Task 1

Take a dataset and:

- clean it
- preprocess it

- compare scaled vs unscaled results

Task 2

Manually calculate:

- precision
- recall
- F1

from a confusion matrix.

PHASE 2 — Classical ML Algorithms (Week 2–3)

This phase is where most “tool users” become actual ML engineers.

You need intuition.

Not memorization.

1. Linear Regression

Learn

- Best fit line
- Cost function
- Gradient descent intuition
- Coefficients
- Residuals

Understand Deeply

Why does gradient descent work?

Understand:

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i)^2$$

This is the error surface the algorithm tries to minimize.

2. Logistic Regression

Most beginners misunderstand this.

It is NOT regression.

It is classification.

Learn

- Sigmoid function
- Probability outputs
- Decision boundary
- Log loss

Understand:

{"math_block_widget_always_prefetch_v2":{"content":"f(x)=\\frac{1}{1+e^{-x}}"}

Why sigmoid converts values into probabilities.

3. KNN

Learn

- Distance metrics
- Choosing K
- Curse of dimensionality

Understand:

- why scaling matters heavily
 - why KNN becomes slow with large datasets
-

4. Decision Trees

VERY important.

Trees are the foundation for:

- Random Forest

- XGBoost
 - LightGBM
 - CatBoost
-

Learn

- Entropy
- Information gain
- Gini impurity
- Tree depth
- Splits

Understand:

- why trees overfit easily
 - why pruning helps
-

5. Random Forest

This is where ensemble learning starts.

Learn

- Bagging
- Bootstrap sampling
- Feature randomness
- Voting

Understand:

- why multiple weak trees become powerful
 - why random forests reduce variance
-

6. XGBoost (VERY IMPORTANT)

You specifically mentioned this.

Most people use it without understanding it.

After this phase, you should fully understand:

- what boosting is
 - why boosting works
 - why XGBoost became famous
-

Learn in Order

Step 1 — Weak Learners

Small trees perform slightly better than random.

Boosting combines MANY weak learners.

Step 2 — Sequential Learning

Unlike Random Forest:

Random Forest:

- trees independent

XGBoost:

- each new tree learns from previous mistakes

That is the KEY IDEA.

Step 3 — Residuals

Each tree tries to predict:

- previous errors
- residuals

This is the core intuition.

Step 4 — Gradient Boosting

Understand:

- loss minimization
 - gradient direction
 - additive learning
-

Step 5 — Why XGBoost Is Powerful

Learn:

- regularization
 - parallel optimization
 - pruning
 - handling missing values
 - speed improvements
-

IMPORTANT COMPARISON

You MUST be able to compare:

Algorithm	Main Idea
Decision Tree	Single tree
Random Forest	Bagging
XGBoost	Boosting

If you understand this table deeply, your ensemble understanding becomes strong.

PHASE 3 — Mathematics for ML (Week 4)

You do NOT need heavy university-level math.

You need PRACTICAL ML math.

1. Linear Algebra

Learn:

- vectors
- matrices
- dot product
- transpose
- matrix multiplication
- dimensions

Understand:

- why features are matrices
- why neural networks use matrix multiplication

2. Statistics & Probability

Learn:

- mean
- variance
- standard deviation
- distributions
- normal distribution
- conditional probability
- Bayes theorem

Useful formula:

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}$$

3. Calculus for ML

ONLY learn what is needed.

Learn:

- slope
- derivatives
- partial derivatives
- chain rule intuition

- gradients

Why?

Because deep learning is basically:

“Using calculus to reduce error.”

IMPORTANT

You do NOT need advanced proofs.

Focus on:

- intuition
 - applications
 - visualization
-

PHASE 4 — Advanced ML Concepts (Week 5)

This phase separates average learners from strong learners.

1. Regularization

Learn deeply:

- L1 regularization
- L2 regularization
- Ridge
- Lasso

Understand:

- why large weights are dangerous
 - how regularization reduces overfitting
-

2. Cross Validation

Learn:

- K-Fold CV
- stratified CV

Understand:

- why one train/test split is unreliable
-

3. Hyperparameter Tuning

Learn:

- GridSearchCV
- RandomizedSearchCV

Understand:

- parameters vs hyperparameters
-

4. Feature Engineering

Learn:

- feature selection
- feature extraction
- interaction features
- polynomial features

This is one of the most important real-world ML skills.

5. Feature Importance

Learn:

- coefficients
- tree feature importance
- SHAP basics

This helps interpret models.

PHASE 5 — Unsupervised Learning (Week 6)

1. K-Means Clustering

Learn:

- centroids
- distance minimization
- elbow method

Understand:

- why initialization matters
-

2. PCA

VERY important.

Learn:

- dimensionality reduction
- variance preservation
- projections

Understand:

- curse of dimensionality
 - compression intuition
-

3. Anomaly Detection

Learn basics.

Very useful in real-world systems.

PHASE 6 — Build Understanding Through Projects (Week 7)

Now revisit your old projects.

This time ask:

- Why did I choose this model?
- Was scaling needed?
- Was there leakage?
- Could another metric work better?
- Was the model overfitting?
- Why did XGBoost outperform others?

This phase is where REAL learning happens.

Mini Projects You Should Build

1. Model Comparison Project

Compare:

- Logistic Regression
- Decision Tree
- Random Forest
- XGBoost

on the same dataset.

Explain:

- speed
 - accuracy
 - overfitting
 - interpretability
-

2. Feature Engineering Project

Improve performance using:

- feature creation

- scaling
 - preprocessing
-

3. End-to-End ML Pipeline

Build:

- preprocessing
- training
- evaluation
- hyperparameter tuning

This is excellent for resumes.

PHASE 7 — Restart Deep Learning (Week 8)

ONLY start after:

- you understand gradient descent
- regularization
- overfitting
- optimization basics
- matrices
- activation intuition

Then Deep Learning becomes MUCH easier.

Before Starting Deep Learning, You MUST Understand

1. Gradient Descent

This is EVERYTHING.

2. Chain Rule Intuition

Backpropagation depends on this.

3. Matrix Multiplication

Neural networks are matrix operations.

4. Activation Functions

- sigmoid
 - tanh
 - ReLU
-

5. Loss Functions

- MSE
 - Binary cross entropy
-

BEST RESOURCES

For Intuition

YouTube

- StatQuest
 - Codebasics
 - 3Blue1Brown
 - Krish Naik
-

For Practical ML

Hands-on

- Kaggle
 - Scikit-learn documentation
-

For Math Intuition

Watch

- Essence of Linear Algebra
- Essence of Calculus

by  entity  ["organization", "3Blue1Brown", "Educational YouTube channel"] .

DAILY STUDY STRUCTURE

2-Hour Example

40 min

Concept learning

30 min

Implementation

20 min

Write notes in your own words

30 min

Mini experiment/project

HOW TO TAKE NOTES

For every algorithm:

Use this template:

1. What problem does it solve?
2. Main intuition
3. Advantages
4. Disadvantages
5. When to use it
6. Important hyperparameters

- 7. Overfitting behavior
- 8. Real-world use cases

This alone will massively improve understanding.

MOST IMPORTANT THINGS TO MASTER

If you master these concepts, Deep Learning becomes MUCH easier:

1. Gradient descent
 2. Bias vs variance
 3. Overfitting
 4. Regularization
 5. Decision trees
 6. Ensemble learning
 7. Feature engineering
 8. Evaluation metrics
 9. Matrix multiplication
 10. Probability basics
-

FINAL ADVICE

Right now, do NOT chase:

- LLMs
- agents
- advanced AI hype
- complicated frameworks

Strengthen fundamentals first.

The people who become genuinely strong in AI/ML are usually the ones with:

- solid basics
- strong intuition
- ability to explain concepts simply

You already have project experience.

Now you need depth.

Once this revision is done:

- your future ML projects improve massively
- Deep Learning becomes easier
- papers/tutorials make more sense
- interviews become easier
- your confidence increases a lot

This revision phase is not “starting over.”

It is upgrading from:

- “using ML tools” to
- “understanding ML systems.”